



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Malaysia-Japan
International
Institute of Technology
(MJIT)

SECJ 3483 – WEB TECHNOLOGY

DAC 3.0 Challenge

Project Name : Resobot
Developer : Md Jubayer Islam
Matric No : A23MJ4007

1. Introduction and Scenario Description

1.1 Overview: This project is developed to solve a scenario problem given by DHL for Challenge 3.0. The system, named ResoBot is designed to automate the reporting system at DHL from multiple source like whatsapp, email, telegram, hand writtent text to a centralized AI powered platform.

1.2 Introduction and Scenario Description: The project tackles Scenario 2: AI-Enhanced Incident Reporting & Resolution System. DHL Customer Support teams process a massive volume of incident reports daily, ranging from late deliveries to damaged parcels. Because these reports originate from disparate and unstructured channels (emails, Telegram chats, images, handwritten notes), DHL faces challenges in identifying root causes, assigning the correct departments, and prioritizing SLA risks.

1.3 Objective & Solution: The goal with ResoBot is to handle reports automatically by collecting them and keeping track without constant manual checks. It combines an RPA setup with an AI web application to get that done. UiPath seems to act as the starting point by scanning different channels for data that is not structured well and sending it over to one main backend.

An AI controller then pulls out key details like priority level or department and turns the info into a proper incident ticket. Human operators can look at all of this through a dashboard that uses role based access so only certain people see certain things. It helps with monitoring and fixing problems.

Some parts of how the data moves between steps feel a bit unclear still. That part is easy to miss at first.

2. System Architecture

The ResoBot system employs a decoupled, modern web architecture to ensure high availability, security, and real-time responsiveness. It satisfies Option 2 of the assignment requirements by utilizing a robust database rather than a mock API.

2.1 Technology:

1. Frontend: React.js (Vite), Tailwind CSS (User Interface & Interaction).
2. Backend: Node.js, Express.js (REST API & Web Server).
3. Database: MongoDB via Mongoose (Data persistence and management).

4. Telecommunications: Socket.io (WebSocket for real-time telemetry).
5. Automation: UiPath Studio (RPA integration).

2.2 Architectural Flow

1. Ingestion (UiPath): The RPA bot listens to a designated Google Drive folder for new unstructured files (images, text, documents).
2. Transmission: UiPath downloads the file and sends a secure HTTP POST request containing the binary payload to the Node.js backend.
3. Processing & AI Classification: The Express server receives the payload. It performs a cryptographic SHA-256 hash check against the database to prevent duplicate processing (enforcing a 14-day deduplication window). It then utilizes an AI controller to parse the unstructured text or image, returning a structured JSON response.
4. Database Storage: The parsed data is saved to MongoDB as a new Incident document.
5. Real-Time Push: The backend emits a Socket.io event. Connected React clients instantly update their dashboards without requiring a manual browser refresh.

3. API and JSON Structure

The backend exposes a secure RESTful API, adhering to JSON formatting for all data exchange. The API supports full CRUD operations required for incident management.

3.1 Primary Endpoints

HTTP Method	Endpoint	Description
GET	/api/incidents	Fetches all incidents. Returns filtered results based on RBAC.
POST	/api/incidents	Creates a manual incident entry via the web form.
PUT	/api/incidents/:id	Updates status, department, and appends the tracking timeline.

DELETE	/api/incidents/:id	Removes an incident (Restricted to Admin role).
POST	/api/uiopath-drive	Secure endpoint specifically for the UiPath bot to POST unstructured files.

3.2 JSON Document Structure (MongoDB Schema)

JSON

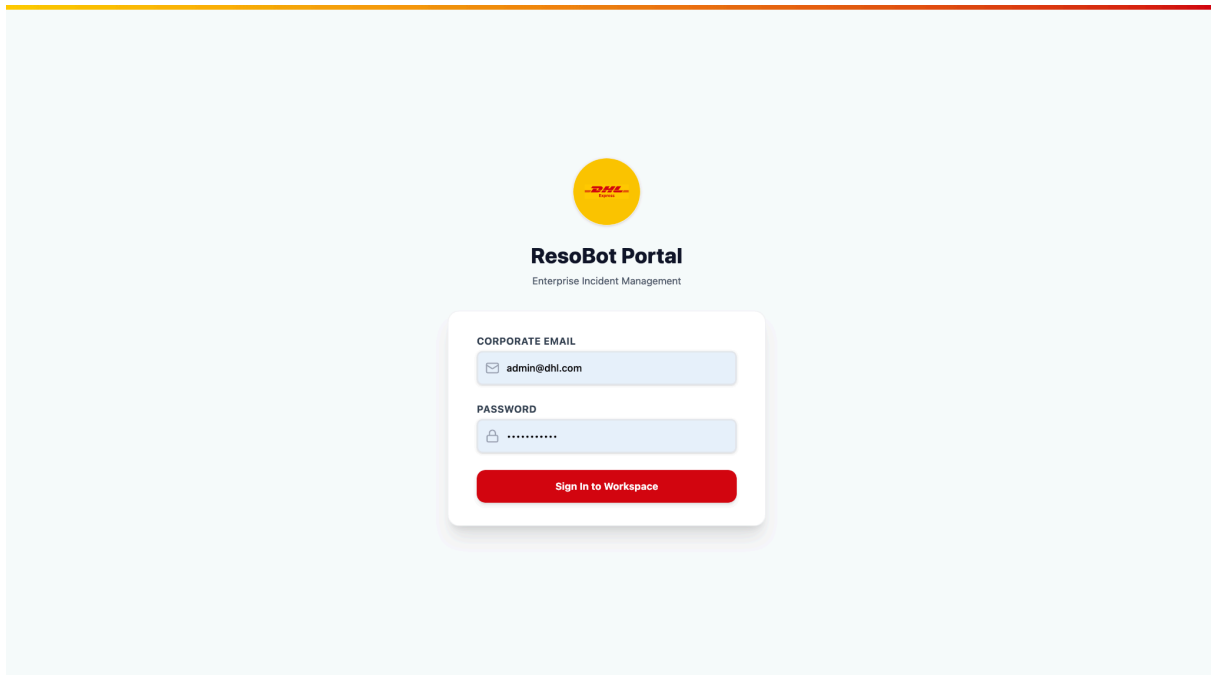
```
{
  "_id": "60d5ecb8b392...",
  "ticketId": "INC-48291",
  "incidentId": "DHL-20260524-8192",
  "source": "image_143b8a.jpg",
  "rawDescription": "Customer reported package arrived wet and torn.",
  "category": "Damaged Goods",
  "priority": "High",
  "department": "Warehouse",
  "aiSummary": "The package sustained water damage and physical tearing during transit.",
  "status": "Reviewed",
  "isBot": true,
  "creator": "UiPath Automation",
  "tracking": [
    {
      "label": "Ingested via Google Drive Bot",
      "timestamp": "May 24, 2026, 12:45 PM",
      "status": "completed",
      "author": "System"
    }
  ],
}
```

```
"createdAt": "2026-05-24T04:45:00.000Z"
```

```
}
```

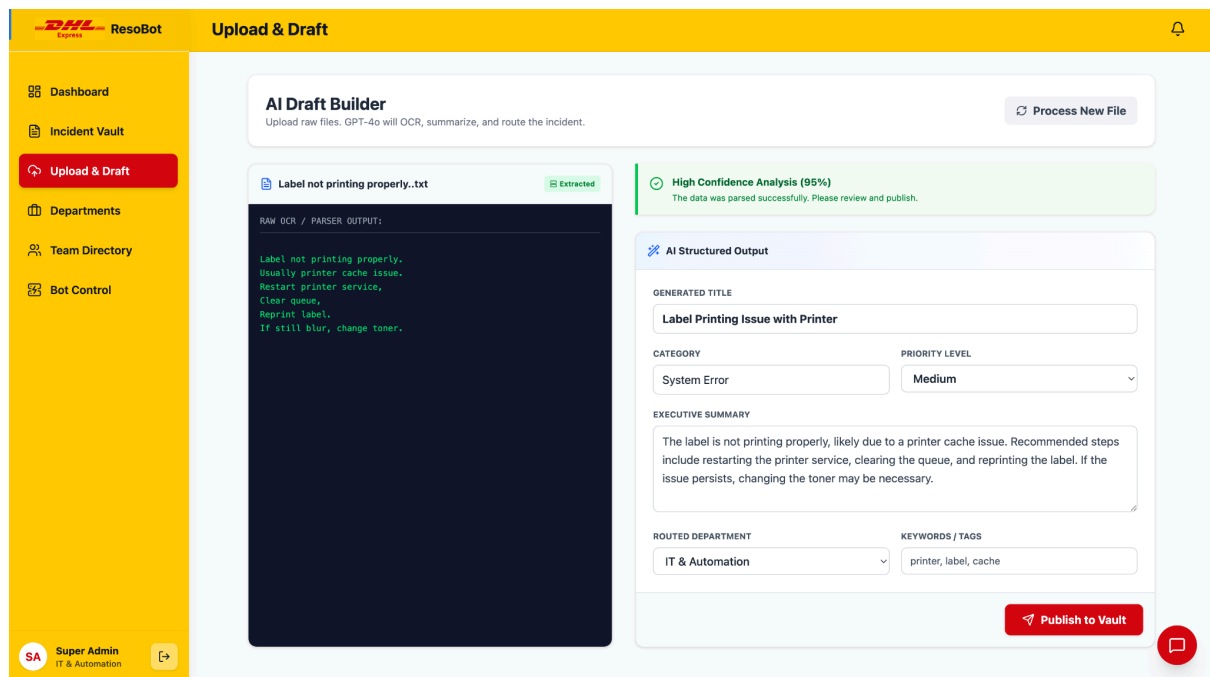
4. User Interface Screenshots

4.1 Login :

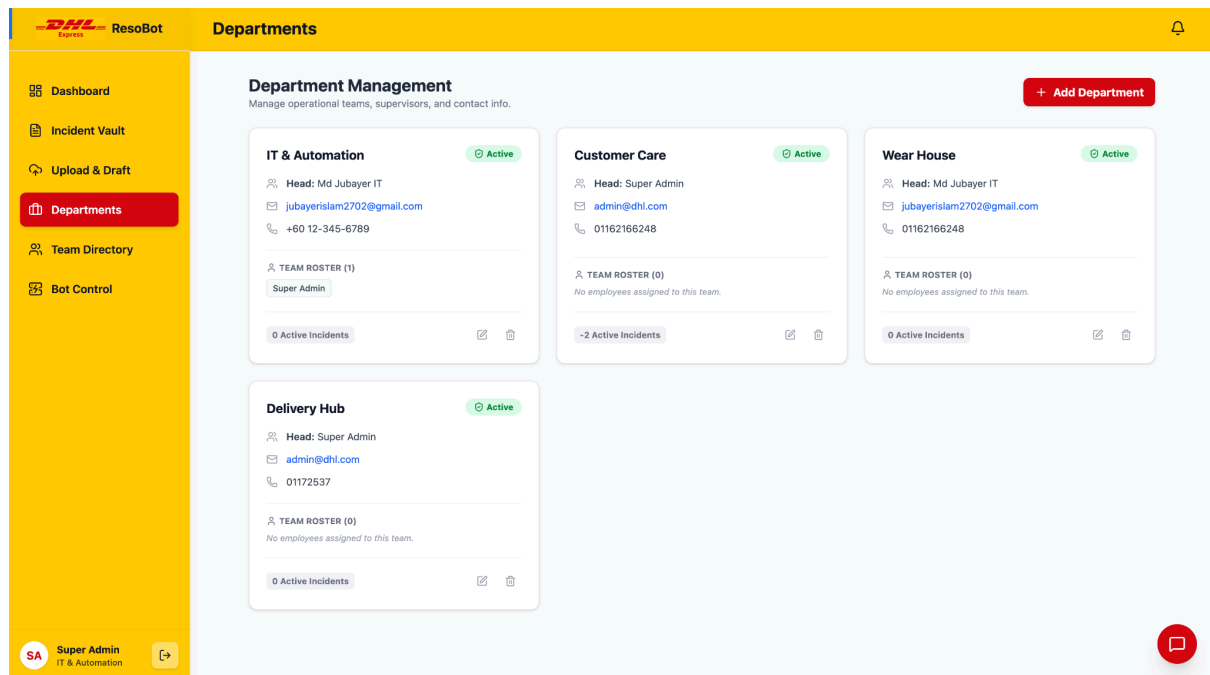


4.2 Dashboard

4.4 Draft Builder

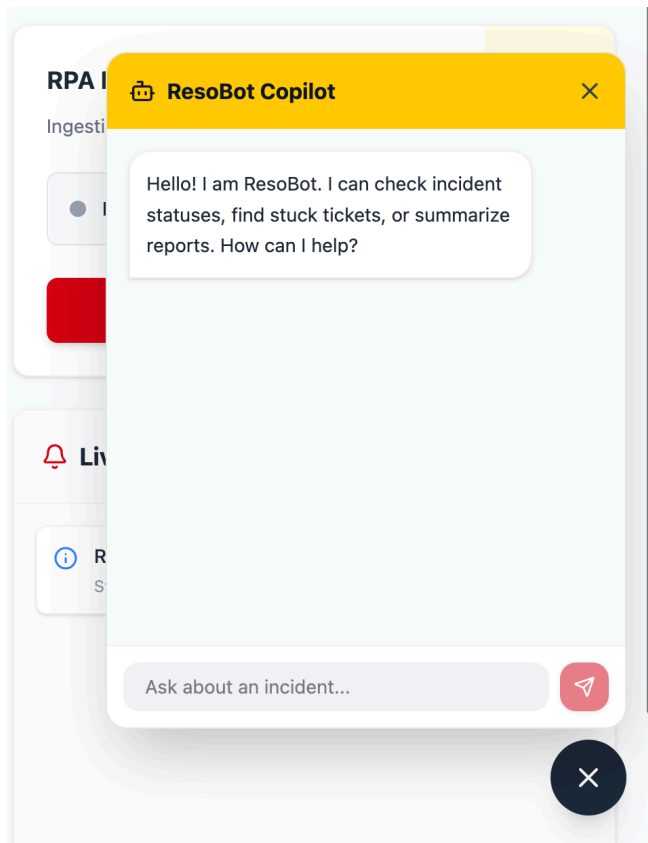


4.5 Department Controls

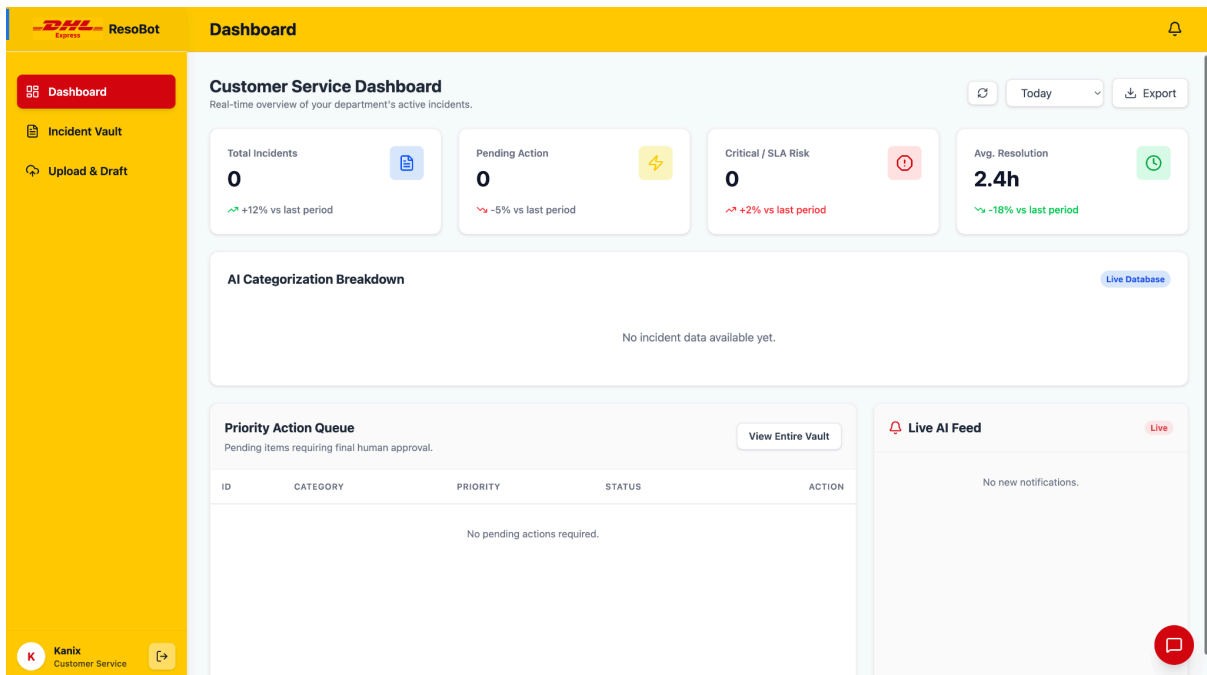


4.6 Team Directory

4.8 Chat Bot



4.9 Employee Dashboard

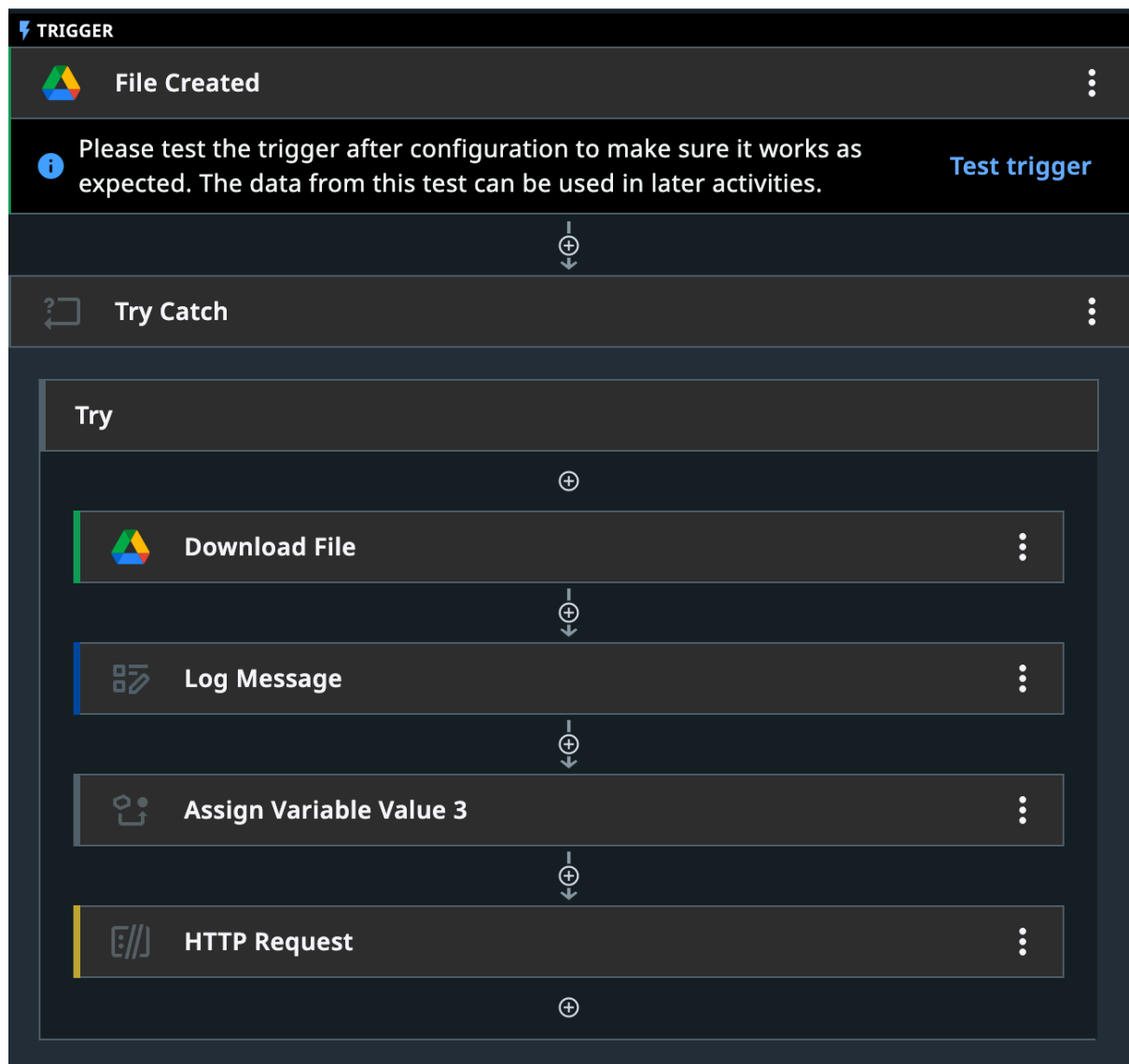


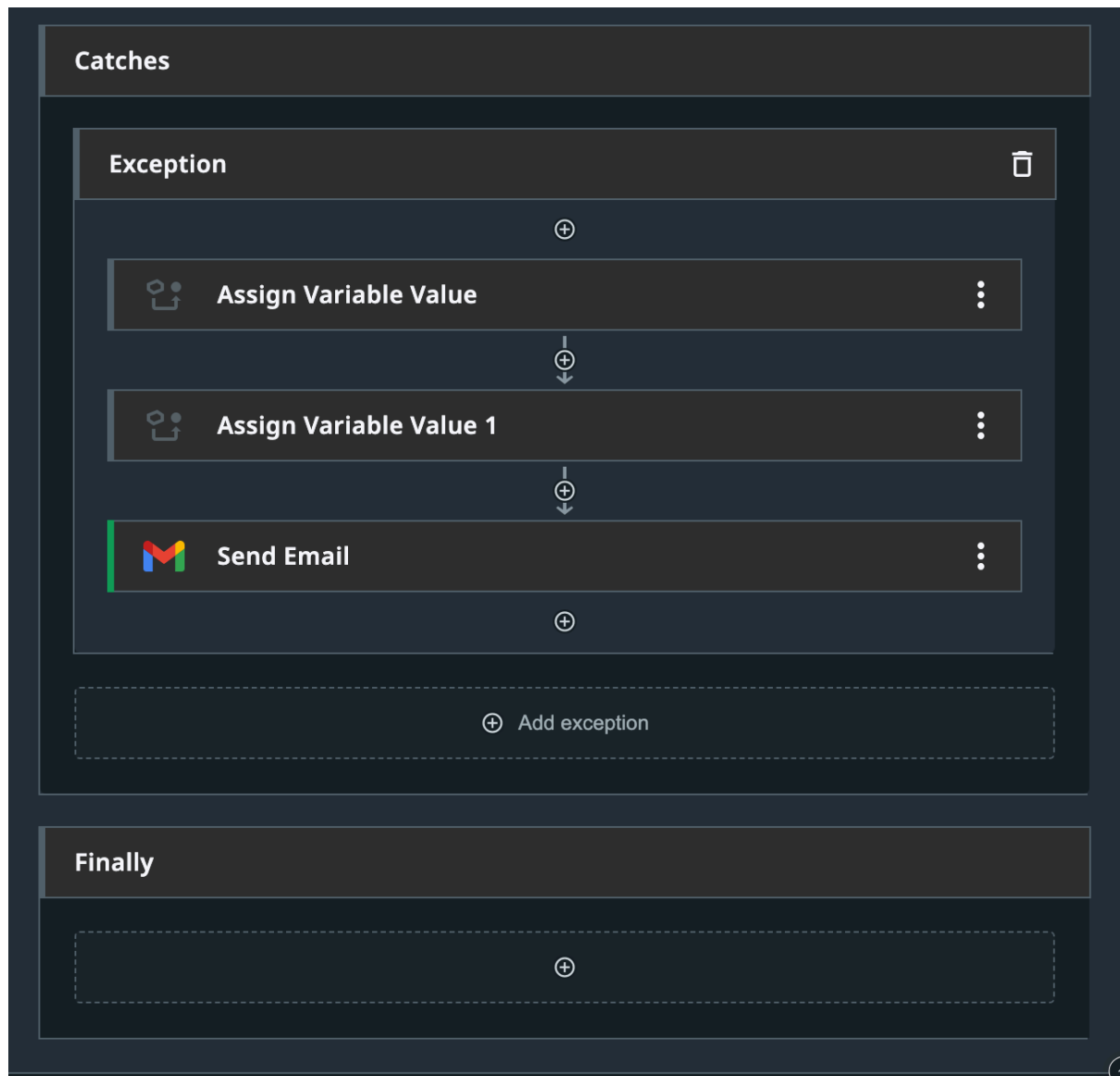
5. RPA Workflow and Automation Explanation

The automation component of this system utilizes UiPath Studio to bridge the gap between unstructured external file repositories and the structured web application environment.

5.1 Process Identification: Checking for new reports in an inbox or Drive folder is a rule-based, repetitive chore that really doesn't need a person to step in until it is time to act on the data. Automating the ingestion part gets rid of manual typing and saves the support teams a lot of time.

5.2 Workflow Design & Logic: The UiPath setup is built to be consistent and resilient by using standard ways to handle errors when they happen.





5.3 Step-by-Step Execution Logic

Phase 1: Automated Triggering

1. **File Created (Trigger):** The workflow operates on an event-driven architecture. A "File Created" trigger continuously monitors a designated Google Drive folder. The moment a new unstructured file (e.g., an incident image or text document) is uploaded, the bot awakens and begins execution.

Phase 2: Primary Execution (The "Try" Block) To ensure maximum resilience, the core ingestion process is wrapped in a **Try Catch** block. The "Try" sequence handles the successful path:

1. **Download File:** The bot securely fetches the newly detected file from the cloud repository to the local execution environment.
2. **Log Message:** The system logs an operational telemetry event to establish a clear audit trail in the orchestrator.
3. **Assign Variable Value :** The bot captures necessary metadata—such as the file path, MIME type, and unique identifiers—and assigns them to local variables to prepare the data payload.
4. **HTTP Request:** The bot formulates a **multipart/form-data** POST request. It securely transmits the physical file and its associated metadata to the Web Application API (**/api/uiopath-drive**) for AI processing and database insertion.

Phase 3: Exception Handling (The "Catches" Block) If any step in the "Try" block fails (e.g., API timeout or network failure), the execution immediately routes to the "Exception" block to prevent data loss:

1. **Assign Variable Values (0 & 1):** The bot catches the specific system exception and uses assign activities to format an error message string and define the target administrator's email address.
2. **Send Email:** A critical alert is automatically dispatched via Gmail to the system administrator, containing the failure details so the IT team can manually intervene and resolve the pipeline blockage.